Synchronous Programmable Counter

David Velasco Perez, velascodavidp@gmail.com

January 28, 2026

1. Synchronous Programmable Counter

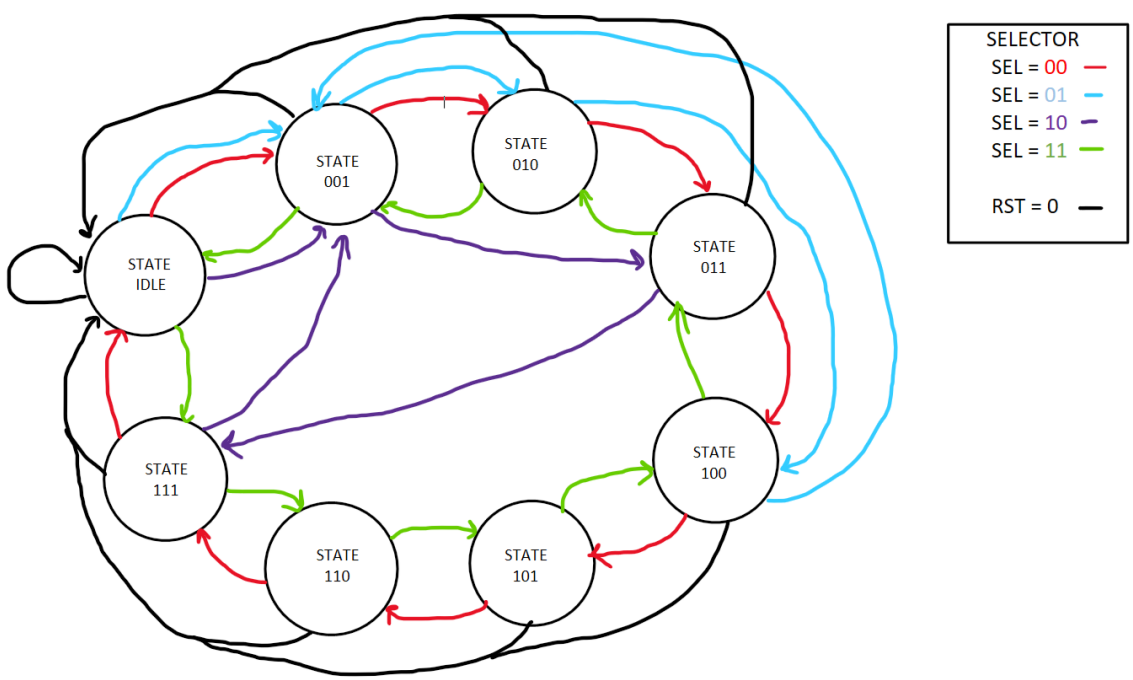
1.1. Module Description:

This Verilog Module implements a 3-bit synchronous programmable counter with an asynchronous active low reset using Finite State Machine structure. The 2-bit selector input programs the output to the desired counting mode; the 4 counting modes are unsigned binary up-count, one-hot binary up-count, thermometer binary up-count, and unsigned binary down-count. The 3-bit counter updates its output on the clocks rising edge.

1.2. Behavior Table

select [1:0]	Description	count [2:0] progression
00	Unsigned binary up-count	000, 001, 010, 011, 100, 101, 110, 111, 000, 001, …
01	One-hot binary up-count	000, 001, 010, 100, 001, 010, 100, 001, …
10	Thermometer binary up-count	000, 001, 011, 111, 001, 011, 111, 001, …
00	Unsigned binary down-count	000, 111, 110, 101, 100, 011, 010, 001, 000, 111, …

1.3. FSM Diagram



1.4. Verilog Module:

```
1 module sync_prog_cnt (
2                     input rst, clk,
3                     input [1:0] sel,
4                     output [2:0] cnt
5
6 );
7
8     // 8 states for each bit
9     parameter   STATE_IDLE = 3'b000;
10    parameter   STATE_001 = 3'b001;
11    parameter   STATE_010 = 3'b010;
12    parameter   STATE_011 = 3'b011;
13    parameter   STATE_100 = 3'b100;
14    parameter   STATE_101 = 3'b101;
15    parameter   STATE_110 = 3'b110;
16    parameter   STATE_111 = 3'b111;
17
18 //=========================STATE Registers==================================
19    reg [2:0] CURR_STATE;
20    reg [2:0] NEXT_STATE;
21 //==========================================================
22
23
24 //=========================RESET H=====================================
25    always @(posedge clk)
26    begin
27        if(~rst)
28            CURR_STATE <= STATE_IDLE;
29        else
30            CURR_STATE <= NEXT_STATE;
31    end
32 //============================END of RESET ================================
33
34 //=========================FSM navigation==================================
35    always @(*)
36    begin
37        NEXT_STATE = CURR_STATE;
38        case(CURR_STATE)
39            //here we see:
40            //sel 11: 000->111
41            //every thing else: 000->001
42            STATE_IDLE: begin
43                if(sel == 2'b11)
44                    NEXT_STATE = STATE_111;
45                else
46                    NEXT_STATE = STATE_001;
47            end
48 // here we see:
49 // sel 10: 001->011
50 // sel 00 & 01: 001-> 010
51 // sel 11 001->000
52            STATE_001: begin
53                if(sel == 2'b10)
54                    NEXT_STATE = STATE_011;
55                else if(sel == 2'b11)
56                    NEXT_STATE = STATE_IDLE;
57                else
58                    NEXT_STATE = STATE_010;
59            end
60 // sel 00: 010 ->011
61 // sel 01: 010 -> 100
62 // sel 10: never in this state
```

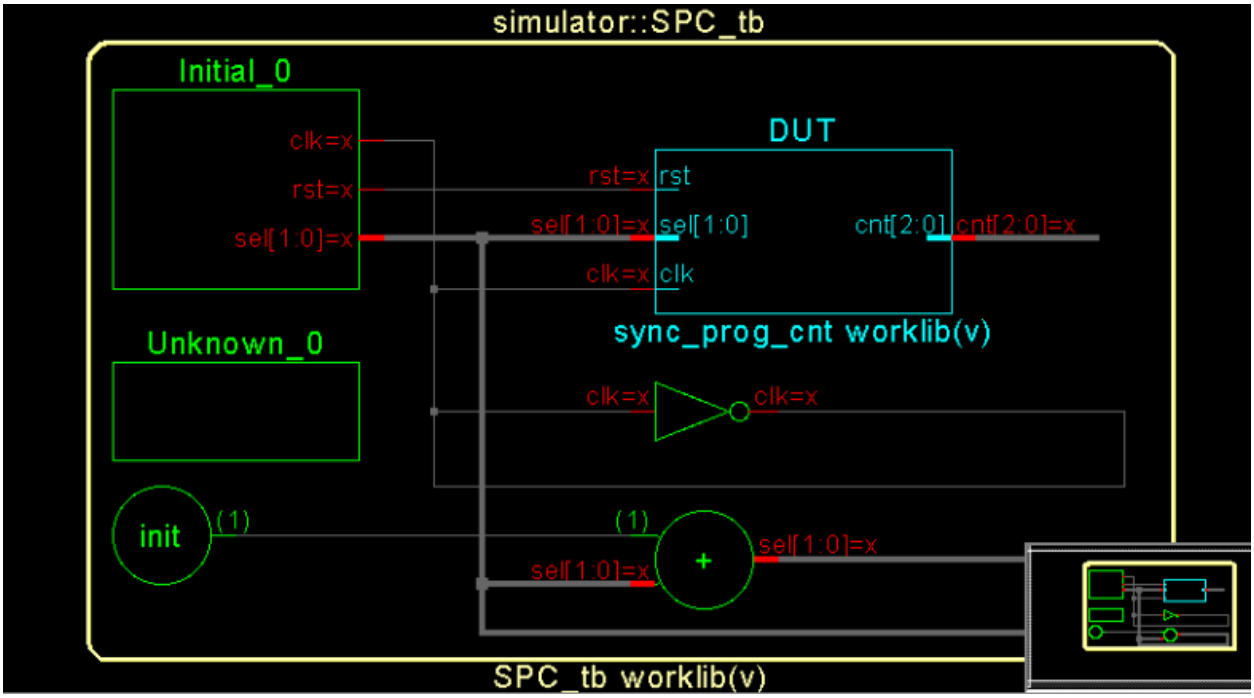
```
// sel 00: 010 ->011
// sel 01: 010 -> 100
// sel 10: never in this state
// sel 11: 010 -> 001
            STATE_010: begin
                if(sel == 2'b00)
                    NEXT_STATE = STATE_011;
                else if (sel == 2'b11)
                    NEXT_STATE = STATE_001;
                else if(sel == 2'b01)
                    NEXT_STATE = STATE_100;
            end
// sel 00: 011 -> 100
// sel 01: never at this stage
// sel 10:  011-> 111
// sel 11: 011 -> 010
            STATE_011: begin
                if(sel == 2'b00)
                    NEXT_STATE = STATE_100;
                else if(sel == 2'b10)
                    NEXT_STATE = STATE_111;
                else if(sel == 2'b11)
                    NEXT_STATE = STATE_010;
            end

//sel 01:100 -> 001
// sel 11 : 100->011
//sel 00: 100->101
//sel 10 never reaches this state
            STATE_100: begin
                if(sel == 2'b00)
                    NEXT_STATE = STATE_101;
                else if(sel == 2'b01)
                    NEXT_STATE = STATE_001;
                else if(sel == 2'b11)
                    NEXT_STATE = STATE_011;

            end

// sel 00:  101->110
// sel 01:  never goes to this stage
// sel 10:  never goes to this stage
// sel 11:  101->100S
            STATE_101: begin
                if(sel == 2'b00)
                    NEXT_STATE = STATE_110;

                else if(sel == 2'b11)
                    NEXT_STATE = STATE_100;
            end

```

```
// sel 00:  110 -> 111
// sel 01: never goes to this stage
// sel 10: never goes to this stage
// sel 11:  110->101
            STATE_110: begin
                if(sel == 2'b00)
                    NEXT_STATE = STATE_111;
                else if(sel == 2'b11)
                    NEXT_STATE = STATE_101;
            end

// sel 00:  111->IDLE
// sel 01: never goes to this stage
// sel 10:  111->001
// sel 11:  111->110
            STATE_111: begin

                    NEXT_STATE = STATE_IDLE;
                else if(sel == 2'b10)
                    NEXT_STATE = STATE_001;
                else if(sel == 2'b11)
                    NEXT_STATE = STATE_110;

            end
            default: NEXT_STATE = STATE_IDLE;
        endcase
    end

//=========================END of FSM navigation==========================

//=========================ASSIGN cnt to the state?=========================
    assign cnt = CURR_STATE;

endmodule
```

1.5. Verilog Testbench:

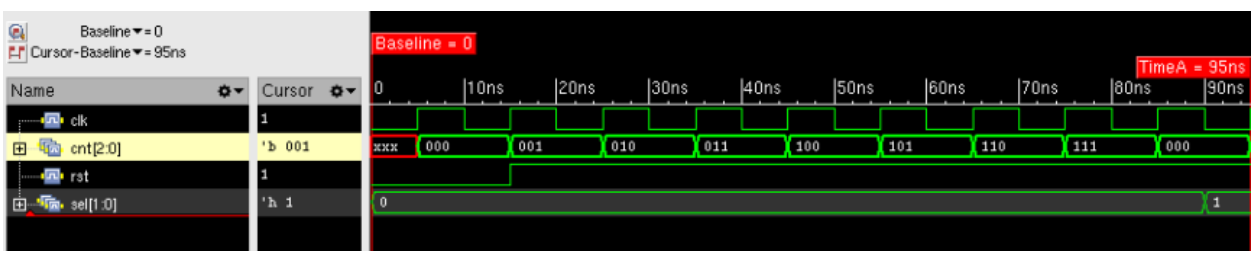
```
// synchronous programable counter test bench
module SPC_tb();

    reg clk,rst;
    reg [1:0] sel;

    wire [2:0] cnt;


    sync_prog_cnt DUT(.rst(rst), .clk(clk), .sel(sel), .cnt(cnt));

    always #5 clk = ~clk;
    always #90 sel = sel+1;
    initial begin
        clk = 0;sel = 2'd0; rst = 0;
        repeat(1) @(posedge clk);
        rst = 1;
        #170 rst = 0;
        #5 rst = 1;
        //#85 rst = 0;
        //#5 rst = 1;
        #600 $finish;
    end
    initial $monitor($time, "clk=%b rst=%b sel=%b cnt=%b", clk, rst, sel, cnt);
endmodule
```

1.6. Xcelium Symbol

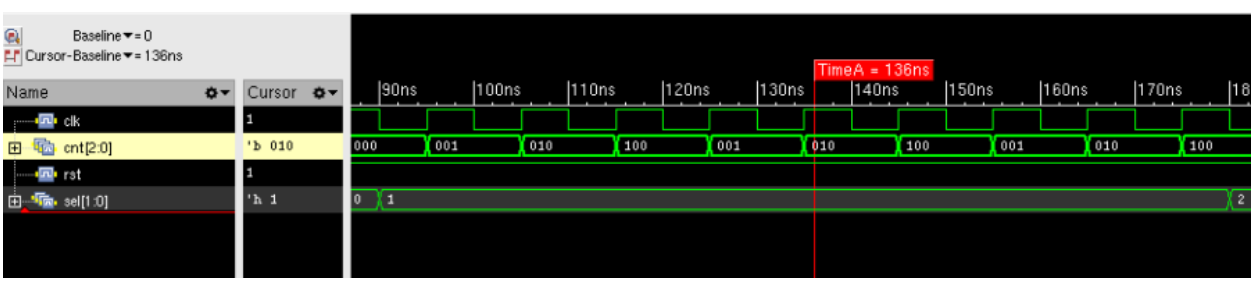


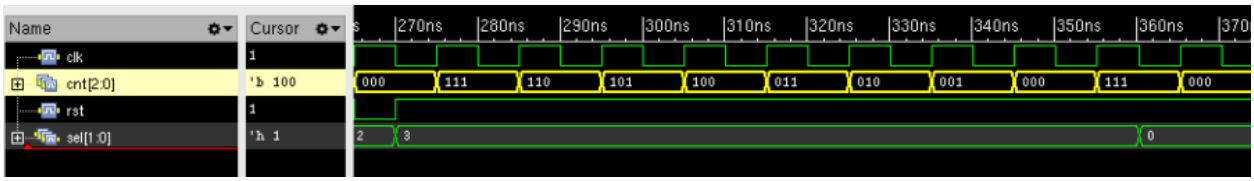
1.7. Simulation Waveforms

1.7.1. Selector = 00



1.8. Selector = 01



1.9. Selector =10

1.10. Selector =11